实验二：词向量的学习与分析

目录

1 实验目的 **2**

2 实验环境 **2**

3 独热表示与分布式表示对比 **2**

3.1 独热表示及其局限性 2

3.2 基于上下文统计的低维表示 2

3.3 修改语料前后的比较 4

4 Skip-gram 训练样本构造 **6**

4.1 准备语料与建立词表 6

4.2 构造中心词—上下文词样本 7

4.3 不同窗口大小的比较 7

4.4 提高任务：构造 CBOW 样本 8

5 模型结构与训练过程 **9**

5.1 Skip-gram 模型结构 9

5.2 训练设置 9

5.3 训练结果 10

6 词向量相似度结果 **11**

6.1 词对余弦相似度 11

6.2 相似词查询结果 11

6.3 独热表示与训练所得词向量对比 12

7 结果分析 **14**

7.1 上下文关系与语义关系 14

7.2 PCA 二维可视化 14

7.3 小语料实验的局限性 14

8 Word2Vec、GloVe、ELMo 比较 **15**

8.1 不同表示方法的区别 15

8.2 完整 Softmax 与负采样 16

8.3 层次 Softmax 与哈夫曼树 16

9 问题记录与解决方法 **16**

9.1 PyTorch 缺失与内核环境问题 16

9.2 PCA 图中文字体缺失 16

9.3 修改语料后的结果覆盖 16

10 实验总结 **17**

1　实验目的

本实验通过独热表示、基于共现统计的低维表示和 Skip-gram 模型，了解词语如何转换为数值向量，以及词向量与上下文之间的关系。实验内容和语料以教师提供的实验设计为依据 [1]。

具体目的包括：理解独热表示的局限性；根据上下文窗口构造训练样本；使用 PyTorch 实现并训练小型 Skip-gram 模型；通过余弦相似度查询相似词；分析语料、窗口和降维对结果的影响。同时，通过构造 CBOW 样本以及比较 Word2Vec、GloVe 和 ELMo，理解不同词表示方法的基本区别。

2　实验环境

实验在 Windows 系统下进行，使用 Anaconda 管理 Python 环境，在 Jupyter Notebook 中编写和运行代码。最终使用的内核为“Python（NLP 实验 2）”，模型在 CPU 上训练。

表 1　实验环境与工具

环境或工具	版本或用途
Python	3.10.22
PyTorch	2.14.1+cpu，用于模型建立与训练
NumPy	2.2.6，用于矩阵运算和 SVD
pandas	2.3.3，用于整理结果表格
Matplotlib	3.10.9，用于绘制损失曲线和散点图
scikit-learn	1.7.2，用于 PCA 降维
Jupyter Notebook	用于代码运行与结果记录

实验语料已通过空格划分词语，因此直接使用 split() 得到词序列。本次没有另外进行 jieba 中文分词，也没有使用预训练词向量。

3　独热表示与分布式表示对比

3.1　独热表示及其局限性

首先选取“猫、狗、鸟、飞机”四个词，按照词表编号建立独热向量。每个词对应一个 4 维向量，仅自身编号对应的位置为 1，其余位置为 0。对于任意两个不同词，其独热向量的内积为 0，欧氏距离为

$$\sqrt{(1-0)^2+(0-1)^2}=\sqrt{2}\approx 1.4142。$$

因此，“猫—狗”和“猫—飞机”具有相同的独热距离及余弦相似度，无法体现两组词在语义或用法上的差别。独热向量的维度还会随词表规模增大，形成高维稀疏表示。

词袋表示通常用词频或词权重描述一段文本，也依赖固定词表，但它主要用于表示文本，与本实验中为单个词建立向量的任务有所不同。

3.2　基于上下文统计的低维表示

实验采用 12 条教学短语，在左右各 2 个词的窗口内统计目标词周围的上下文。四个目标词作为矩阵行，上下文词作为矩阵列，得到一个 4×14 的共现矩阵，构造过程和输出如图1所示。

手工检查可知，“猫—可爱”的计数为 1，“猫—宠物”和“狗—宠物”的计数均为

1，与程序输出一致。随后，对共现矩阵逐行进行归一化，使用奇异值分解（SVD）保留前三个方向，得到 4×3 的低维词向量。

```
[9]: demo_sentences = [
         "可爱 猫 宠物", "可爱 狗 宠物",
         "家中 猫 陪伴", "家中 狗 陪伴",
         "猫 吃 鱼", "狗 吃 肉",
         "鸟 天空 飞翔", "鸟 树林 飞翔",
         "飞机 天空 飞行", "飞机 机场 飞行",
         "鸟 空中 移动", "飞机 空中 移动",
     ]
     demo_tokens = [s.split() for s in demo_sentences]
     demo_contexts = sorted({w for sent in demo_tokens for w in sent} - set(demo_words))
     demo_context_ids = {w: i for i, w in enumerate(demo_contexts)}
     demo_counts = np.zeros((len(demo_words), len(demo_contexts)), dtype=float)
     for sent in demo_tokens:
         for pos, word in enumerate(sent):
             if word not in demo_ids:
                 continue
             for j in range(max(0, pos - 2), min(len(sent), pos + 3)):
                 if j != pos and sent[j] in demo_context_ids:
                     demo_counts[demo_ids[word], demo_context_ids[sent[j]]] += 1
     print(pd.DataFrame(demo_counts, index=demo_words, columns=demo_contexts).to_string())
```

```
      可爱   吃   天空   宠物   家中   机场   树林   移动   空中   肉   陪伴   飞翔   飞行   鱼
猫   1.0  1.0  0.0  1.0  1.0  0.0  0.0  0.0  0.0  0.0  1.0  0.0  0.0  1.0
狗   1.0  1.0  0.0  1.0  1.0  0.0  0.0  0.0  0.0  1.0  1.0  0.0  0.0  0.0
鸟   0.0  0.0  1.0  0.0  0.0  0.0  1.0  1.0  1.0  0.0  0.0  2.0  0.0  0.0
飞机 0.0  0.0  1.0  0.0  0.0  1.0  0.0  1.0  1.0  0.0  0.0  0.0  2.0  0.0
```

图 1　原始教学语料及其共现矩阵

```
        维度1      维度2     维度3
猫    0.9574   0.0000   0.000
狗    0.9574   0.0000   0.000
鸟    0.0000   0.8292  -0.559
飞机   0.0000   0.8292   0.559
独热表示形状:  (4, 4)
分布式表示形状:  (4, 3)
```

(a) 原始 SVD 词向量及表示形状

```
独热表示的余弦相似度矩阵:
      猫    狗    鸟    飞机
猫   1.0   0.0   0.0   0.0
狗   0.0   1.0   0.0   0.0
鸟   0.0   0.0   1.0   0.0
飞机  0.0   0.0   0.0   1.0
分布式表示的余弦相似度矩阵:
      猫    狗     鸟      飞机
猫   1.0   1.0   0.000   0.000
狗   1.0   1.0   0.000   0.000
鸟   0.0   0.0   1.000   0.375
飞机  0.0   0.0   0.375   1.000
```

(b) 两种表示的余弦相似度矩阵

图 2　原始语料下的低维表示与相似度矩阵

原始语料下的词对指标见表2，相应运行输出见图3。

原始短语中，“猫”和“狗”共享“可爱、宠物、家中、陪伴、吃”等上下文，因此具有相近的上下文分布。它们在“鱼”和“肉”列仍有差别，但截断为 3 维后，区分这部分差异的方向没有被保留，最终向量重合。这不意味着两个词完全同义。

“鸟”和“飞机”共享“天空、空中、移动”等上下文，因此相似度高于部分其他词对，但二者分别属于动物和交通工具，也不是同义词。

本步骤用于演示上下文统计如何形成低维表示，并非 Word2Vec 或 GloVe 的实现。

表 2　原始语料下独热表示与 SVD 表示的词对指标

词对	独热 欧氏距离	独热 余弦相似度	SVD 欧氏距离	SVD 余弦相似度
猫—狗	1.4142	0.0000	0.0000	1.0000
猫—鸟	1.4142	0.0000	1.3844	0.0000
猫—飞机	1.4142	0.0000	1.3844	0.0000
狗—鸟	1.4142	0.0000	1.3844	0.0000
狗—飞机	1.4142	0.0000	1.3844	0.0000
鸟—飞机	1.4142	0.0000	1.1180	0.3750

```
词对    独热_欧氏距离    独热_余弦相似度    分布式_欧氏距离    分布式_余弦相似度
猫–狗    1.4142          0.0     0.0000        1.000
猫–鸟    1.4142          0.0     1.3844        0.000
猫–飞机   1.4142          0.0     1.3844        0.000
狗–鸟    1.4142          0.0     1.3844        0.000
狗–飞机   1.4142          0.0     1.3844        0.000
鸟–飞机   1.4142          0.0     1.1180        0.375
```

图 3　原始语料下六组词对的指标输出

向量的各个维度也不能直接命名为“动物性”“飞行性”等语义属性。

3.3　修改语料前后的比较

按照实验要求，将一条关于“狗”的短语由“可爱狗宠物”替换为“狗天空飞行”。其他短语、窗口大小和 SVD 保留维数不变，再重新统计共现矩阵、计算词向量及相似度。整个过程中只修改输入语料，没有直接修改向量数值。

表 3　修改语料前后的 SVD 余弦相似度比较

词对	修改前余弦相似度	修改后余弦相似度
猫—狗	1.0000	0.6937
猫—鸟	0.0000	0.0247
猫—飞机	0.0000	–0.0934
狗—鸟	0.0000	0.1226
狗—飞机	0.0000	0.6222
鸟—飞机	0.3750	0.4193

修改后，“狗”在“可爱”和“宠物”列的计数由 1 变为 0，在“天空”和“飞行”列的计数由 0 变为 1。“猫—狗”的余弦相似度下降约 0.3063，而“狗—飞机”的相似度上升。

这一结果说明，基于上下文统计得到的词表示会随语料变化。由于 SVD 对整个矩阵进行分解，修改一行还可能影响其他词的低维表示。该实验使用经过设计的短语语料，只能说明当前统计关系，不能证明词向量在真实语言应用中的可靠性。

```
[15]: demo_sentences = [
          "可爱 猫 宠物", "狗 天空 飞行",
          "家中 猫 陪伴", "家中 狗 陪伴",
          "猫 吃 鱼", "狗 吃 肉",
          "鸟 天空 飞翔", "鸟 树林 飞翔",
          "飞机 天空 飞行", "飞机 机场 飞行",
          "鸟 空中 移动", "飞机 空中 移动",
      ]
      demo_tokens = [s.split() for s in demo_sentences]
      demo_contexts = sorted({w for sent in demo_tokens for w in sent} - set(demo_words))
      demo_context_ids = {w: i for i, w in enumerate(demo_contexts)}
      demo_counts = np.zeros((len(demo_words), len(demo_contexts)), dtype=float)
      for sent in demo_tokens:
          for pos, word in enumerate(sent):
              if word not in demo_ids:
                  continue
              for j in range(max(0, pos - 2), min(len(sent), pos + 3)):
                  if j != pos and sent[j] in demo_context_ids:
                      demo_counts[demo_ids[word], demo_context_ids[sent[j]]] += 1
      print(pd.DataFrame(demo_counts, index=demo_words, columns=demo_contexts).to_string())
```

```
      可爱   吃   天空   宠物   家中   机场   树林   移动   空中   肉   陪伴   飞翔   飞行   鱼
猫   1.0  1.0  0.0  1.0  1.0  0.0  0.0  0.0  0.0  0.0  1.0  0.0  0.0  1.0
狗   0.0  1.0  1.0  0.0  1.0  0.0  0.0  0.0  0.0  1.0  1.0  0.0  1.0  0.0
鸟   0.0  0.0  1.0  0.0  0.0  0.0  1.0  1.0  1.0  0.0  0.0  2.0  0.0  0.0
飞机  0.0  0.0  1.0  0.0  0.0  1.0  0.0  1.0  1.0  0.0  0.0  0.0  2.0  0.0
```

图 4　将一条“狗”的短语改为飞行相关短语后重新统计共现矩阵

```
      维度1     维度2     维度3
猫   0.5448 -0.7167  0.3256
狗   0.8447 -0.3056 -0.2062
鸟   0.5033  0.6204  0.5960
飞机  0.7153  0.4704 -0.4239
独热表示形状：  (4, 4)
分布式表示形状：  (4, 3)
```

(a) 修改后的 SVD 词向量

```
独热表示的余弦相似度矩阵：
      猫    狗    鸟   飞机
猫   1.0  0.0  0.0  0.0
狗   0.0  1.0  0.0  0.0
鸟   0.0  0.0  1.0  0.0
飞机  0.0  0.0  0.0  1.0
分布式表示的余弦相似度矩阵：
         猫       狗       鸟       飞机
猫   1.0000  0.6937  0.0247 -0.0934
狗   0.6937  1.0000  0.1226  0.6222
鸟   0.0247  0.1226  1.0000  0.4193
飞机 -0.0934  0.6222  0.4193  1.0000
```

(b) 修改后的余弦相似度矩阵

图 5　修改语料后重新计算的词向量与相似度矩阵

词对	独热_欧氏距离	独热_余弦相似度	分布式_欧氏距离	分布式_余弦相似
猫–狗	1.4142	0.0	0.7360	0.6937
猫–鸟	1.4142	0.0	1.3648	0.0247
猫–飞机	1.4142	0.0	1.4142	-0.0934
狗–鸟	1.4142	0.0	1.2718	0.1226
狗–飞机	1.4142	0.0	0.8163	0.6222
鸟–飞机	1.4142	0.0	1.0524	0.4193

图 6　修改语料后的六组词对指标输出

4　Skip-gram 训练样本构造

4.1　准备语料与建立词表

Skip-gram 实验使用 8 个句子，例如“猫喜欢吃鱼”“狗喜欢吃肉”“飞机在天空飞行”等。语料共包含 37 个词次，去重后得到 23 个词。

使用 Counter 统计词频，根据词频建立词表，再构造词语与编号之间的双向映射。训练时使用词编号作为模型输入，输出检查时再将编号转换为词语。准备语料与建立词表的实际运行记录见图7。

这里的语料与任务 1 的 12 条短语不同。修改任务 1 中关于“狗”的短语，没有改变 Skip-gram 的这 8 个训练句子。

```
sentences = [
    "猫 喜欢 吃 鱼",
    "狗 喜欢 吃 肉",
    "猫 和 狗 都 是 宠物",
    "小猫 是 可爱 的 宠物",
    "小狗 是 忠诚 的 宠物",
    "鱼 生活 在 水中",
    "鸟 喜欢 在 天空 飞翔",
    "飞机 在 天空 飞行",
]

corpus = " ".join(sentences).split()
print("语料词数：", len(corpus))
print("前20个词：", corpus[:20])
```

```
语料词数： 37
前20个词： ['猫', '喜欢', '吃', '鱼', '狗', '喜欢', '吃', '肉', '猫', '和', '狗', '都', '是', '宠
物', '小猫', '是', '可爱', '的', '宠物', '小狗']
```

```
from collections import Counter

word_counts = Counter(corpus)
vocab = sorted(word_counts, key=word_counts.get, reverse=True)
word_to_id = {word: i for i, word in enumerate(vocab)}
id_to_word = {i: word for word, i in word_to_id.items()}

print("词表大小：", len(vocab))
print("词表：", vocab)
```

```
词表大小： 23
词表： ['喜欢', '是', '宠物', '在', '猫', '吃', '鱼', '狗', '的', '天空', '肉', '和', '都', '小猫',
'可爱', '小狗', '忠诚', '生活', '水中', '鸟', '飞翔', '飞机', '飞行']
```

图 7　Skip-gram 训练语料与 23 词词表的构建结果

4.2　构造中心词—上下文词样本

Skip-gram 的任务是给定中心词，预测窗口内的上下文词。窗口大小为 1 表示分别查看中心词左侧和右侧最多一个词。对于“猫喜欢吃鱼”，可以生成以下 6 个样本：

```
猫 -> 喜欢
喜欢 -> 猫
喜欢 -> 吃
吃 -> 喜欢
吃 -> 鱼
鱼 -> 吃
```

程序对每个句子分别遍历，跳过中心词自身，将中心词编号和上下文词编号保存为一个训练样本。原始语料在窗口大小为 1 时共生成 58 个样本，见图8。中心词张量和上下文词张量的形状均为 torch.Size([58])，分别作为模型输入和目标标签。

```
def make_skipgram_pairs(tokens, word_to_id, window_size=1):
    pairs = []
    for center_pos, center_word in enumerate(tokens):
        center_id = word_to_id[center_word]
        left = max(0, center_pos - window_size)
        right = min(len(tokens), center_pos + window_size + 1)

        for context_pos in range(left, right):
            if context_pos == center_pos:
                continue
            context_word = tokens[context_pos]
            pairs.append((center_id, word_to_id[context_word]))
    return pairs

sentence_tokens = [sentence.split() for sentence in sentences]
pairs = [pair for tokens in sentence_tokens
         for pair in make_skipgram_pairs(tokens, word_to_id, window_size=1)]

print("训练样本数：", len(pairs))
for center_id, context_id in pairs[:10]:
    print(id_to_word[center_id], "→", id_to_word[context_id])
```

```
训练样本数： 58
猫 → 喜欢
喜欢 → 猫
喜欢 → 吃
吃 → 喜欢
吃 → 鱼
鱼 → 吃
狗 → 喜欢
喜欢 → 狗
喜欢 → 吃
吃 → 喜欢
```

图 8　Skip-gram 样本构造函数与前十个训练样本

4.3　不同窗口大小的比较

窗口增大后，每个中心词能够关联更远的上下文词，因此样本数量增加。较小窗口主要反映近邻搭配；较大窗口可以纳入更宽范围的信息，但也可能包含与中心词关系较弱的词。窗口选择应结合语料和任务目的。

如果将全部句子拼接后直接构造窗口，前一句末尾的词可能与后一句开头的词形成

表 4　不同上下文窗口对应的样本数量

窗口大小	训练样本数
1	58
2	100
3	126

```
for window_size in [1, 2, 3]:
    current_pairs = [pair for tokens in sentence_tokens
                     for pair in make_skipgram_pairs(tokens, word_to_id, window_size)]
    print(f"窗口大小={window_size}, 样本数={len(current_pairs)}")
```

```
窗口大小=1, 样本数=58
窗口大小=2, 样本数=100
窗口大小=3, 样本数=126
```

图 9　窗口大小为 1、2、3 时的样本数量输出

不合理的训练关系。本实验逐句构造样本，避免了跨句子边界的问题。本次只比较了三个窗口下的样本数量，实际模型使用窗口大小 1，没有分别训练三个窗口模型。

4.4　提高任务：构造 CBOW 样本

连续词袋模型（CBOW）使用上下文词预测中心词，与 Skip-gram 的预测方向相反。例如，中心词为“喜欢”时，可以生成 [猫, 吃] → 喜欢。

在样本构造函数中，先收集当前中心词周围的上下文编号，再与中心词编号组成一个样本，即 (context_word_ids, center_word_id)。

窗口大小为 1 时，原始语料共生成 37 个 CBOW 样本。句首和句尾的上下文列表可能只有一个词，句中则通常有两个词。修改后的函数与样本输出如图10所示。

如果进一步建立 CBOW 模型，可以对多个上下文词向量求和或取平均，形成用于预测中心词的输入表示。本次提高任务完成了样本构造，没有进行 CBOW 模型训练，因此不能据此比较两种模型的损失或词向量质量。

在小语料中，两种方法都受到上下文不足和低频词样本较少的影响；扩大语料后，可以进一步比较训练时间、相似词结果和不同词频下的表现。

```python
def make_cbow_samples(tokens, word_to_id, window_size=1):
    samples = []
    for center_pos, center_word in enumerate(tokens):
        center_id = word_to_id[center_word]
        left = max(0, center_pos - window_size)
        right = min(len(tokens), center_pos + window_size + 1)

        context_ids = []
        for context_pos in range(left, right):
            if context_pos == center_pos:
                continue
            context_word = tokens[context_pos]
            context_ids.append(word_to_id[context_word])

        if context_ids:
            samples.append((context_ids, center_id))
    return samples


cbow_samples = [sample for tokens in sentence_tokens
                for sample in make_cbow_samples(
                    tokens, word_to_id, window_size=1)]

print("CBOW训练样本数：", len(cbow_samples))
for context_ids, center_id in cbow_samples[:10]:
    context_words = [id_to_word[i] for i in context_ids]
    print(context_words, "→", id_to_word[center_id])
```

```
CBOW训练样本数： 37
['喜欢'] → 猫
['猫', '吃'] → 喜欢
['喜欢', '鱼'] → 吃
['吃'] → 鱼
['喜欢'] → 狗
['狗', '吃'] → 喜欢
['喜欢', '肉'] → 吃
['吃'] → 肉
['和'] → 猫
['猫', '狗'] → 和
```

图 10　CBOW 样本构造函数及其 37 个样本中的前十项

5　模型结构与训练过程

5.1　Skip-gram 模型结构

使用 PyTorch 建立由 Embedding 层和线性输出层组成的 Skip-gram 模型。Embedding 层根据中心词编号，从词向量矩阵中查找对应的 10 维向量。线性层再将这个向量映射为 23 个输出分数，对应词表中的 23 个候选上下文词。

模型流程为

中心词编号 → Embedding 查表 → 10 维词向量 → 线性层 → 上下文词预测。

Embedding 参数矩阵的形状为 23×10。模型返回未经归一化的输出分数，交叉熵损失函数完成对应的概率计算和损失计算，因此没有在模型中额外添加 Softmax。

5.2　训练设置

每轮训练依次进行梯度清零、前向计算、损失计算、反向传播和参数更新，并保存损失值。每 200 轮输出一次训练日志。

表 5　Skip-gram 模型的训练参数

参数	设置
词表大小	23
词向量维度	10
上下文窗口	左右各 1 个词
训练样本数	58
随机种子	42
优化器	Adam
学习率	0.03
损失函数	CrossEntropyLoss
训练轮数	1000
输出方式	完整 Softmax
每轮训练方式	使用全部 58 个样本

5.3　训练结果

表 6　训练过程中每 200 轮记录的损失

训练轮数	损失
200	0.9814
400	0.9808
600	0.9806
800	0.9805
1000	0.9805

损失曲线在训练初期快速下降，随后逐渐稳定。到第 1000 轮时，损失约为 0.9805，说明模型在当前训练语料上已经趋于收敛。

损失没有降为 0 并不意味着训练失败。同一个中心词可能对应多个上下文词，模型需要学习这些目标的概率分布。损失下降表明模型拟合了部分上下文关系，但还不能说明词向量具有可靠的通用语义能力。

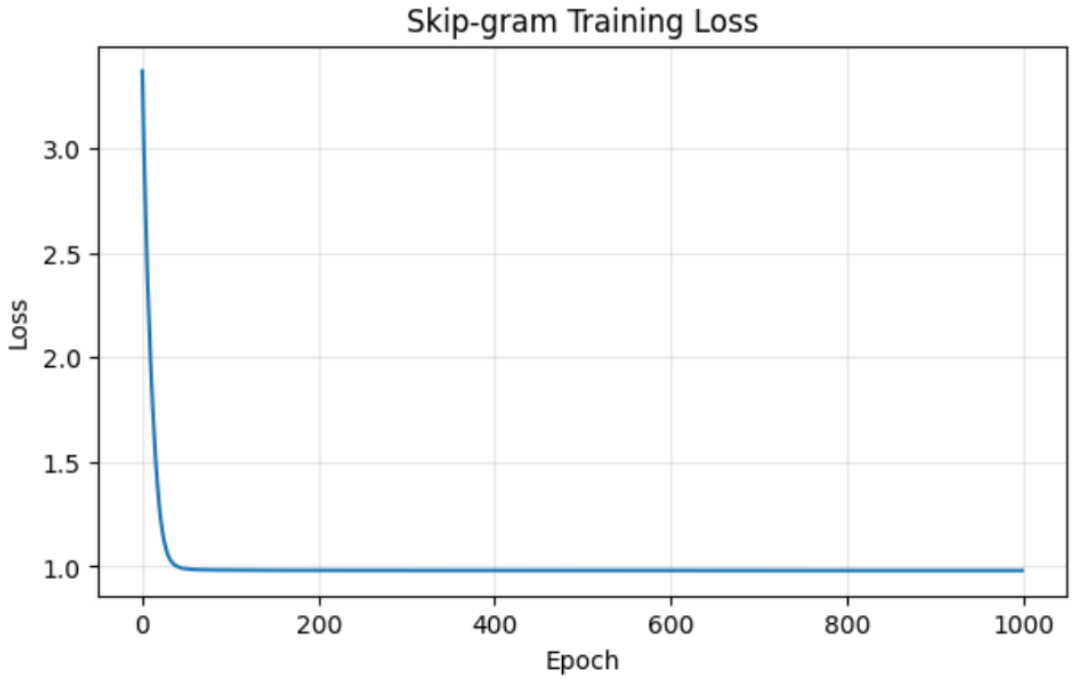


图 11　Skip-gram 模型训练 1000 轮的损失曲线

6　词向量相似度结果

6.1　词对余弦相似度

训练完成后，提取 Embedding 层参数，得到形状为 torch.Size([23, 10]) 的词向量矩阵。使用余弦相似度比较词向量方向：

$$\cos(a, b) = \frac{a^{\mathrm{T}} b}{\|a\|_2 \|b\|_2}。$$

四组词对的结果如表7所示，提取词向量与计算相似度的运行记录见图12。

表 7　四组词对的 Skip-gram 余弦相似度

词对	余弦相似度
猫—狗	0.7484
天空—飞机	0.5342
猫—飞机	0.3546
鱼—水中	−0.3721

在本次结果中，“猫—狗”的相似度高于“猫—飞机”，符合“猫”和“狗”具有部分相似上下文的情况。“鱼—水中”的结果为负，说明现实中的关联不一定直接表现为当前模型中较高的余弦相似度。

6.2　相似词查询结果

对所有词向量计算与查询词的余弦相似度，按相似度降序排列，排除查询词自身，保留前 5 个结果。

“猫”和“狗”互为最相似词。“宠物”的相似词中出现“可爱、忠诚、小猫、小狗”，与训练语料中的宠物相关句子存在联系。同时，结果中也出现“在”“都”等词，说明模型学到的关系受到当前语料分布影响，不完全等同于日常理解中的语义相似。

```
embeddings = model.embedding.weight.detach()
print("词向量矩阵形状：", embeddings.shape)
```

```
词向量矩阵形状： torch.Size([23, 10])
```

```
import torch.nn.functional as F

def cosine_similarity(word_a, word_b):
    vec_a = embeddings[word_to_id[word_a]].unsqueeze(0)
    vec_b = embeddings[word_to_id[word_b]].unsqueeze(0)
    return F.cosine_similarity(vec_a, vec_b).item()

for pair in [("猫", "狗"), ("天空", "飞机"), ("猫", "飞机"), ("鱼", "水中")]:
    print(pair, cosine_similarity(*pair))
```

```
('猫', '狗') 0.7484437823295593
('天空', '飞机') 0.5342141389846802
('猫', '飞机') 0.354570209980011
('鱼', '水中') -0.37214502692222595
```

图 12　词向量矩阵提取与四组词对的余弦相似度计算

表 8　四个查询词的前五个相似词

查询词	前 5 个相似词及相似度
猫	狗 0.7484、鸟 0.4791、在 0.4497、飞翔 0.3977、飞行 0.3627
狗	猫 0.7484、鸟 0.3281、忠诚 0.2171、飞翔 0.2064、在 0.1795
天空	水中 0.6923、飞机 0.5342、鸟 0.3054、生活 0.2645、喜欢 0.1434
宠物	可爱 0.8576、忠诚 0.7224、小猫 0.5098、小狗 0.4704、都 0.2354

6.3　独热表示与训练所得词向量对比

再次选取“猫、狗、鸟、飞机”，比较 23 维独热向量与 10 维 Skip-gram 词向量，结果见表9和图14。

表 9　独热表示与训练所得 Skip-gram 表示的词对指标

词对	独热 欧氏距离	独热 余弦相似度	Skip-gram 欧氏距离	Skip-gram 余弦相似度
猫—狗	1.4142	0.0000	4.2782	0.7484
猫—鸟	1.4142	0.0000	5.8127	0.4791
猫—飞机	1.4142	0.0000	6.3371	0.3546
狗—鸟	1.4142	0.0000	5.9453	0.3281
狗—飞机	1.4142	0.0000	6.4825	0.1668
鸟—飞机	1.4142	0.0000	6.6240	−0.0466

独热表示仍然无法区分不同词对，而训练所得表示给出了不同的距离和相似度。本次 Skip-gram 结果中，“猫—狗”的欧氏距离是这 6 组词对中最小的，余弦相似度最高。

```python
def most_similar(word, top_k=5):
    target_id = word_to_id[word]
    target = embeddings[target_id].unsqueeze(0)
    similarities = F.cosine_similarity(target, embeddings)

    result = []
    for idx in torch.argsort(similarities, descending=True):
        idx = idx.item()
        if idx == target_id:
            continue
        result.append((id_to_word[idx], float(similarities[idx])))
        if len(result) >= top_k:
            break
    return result

for word in ["猫", "狗", "天空", "宠物"]:
    print(f"{word} 的相似词：", most_similar(word))
```

```
猫 的相似词： [('狗', 0.7484437823295593), ('鸟', 0.4791223406791687), ('在', 0.44971176981925964), ('飞翔', 0.39766237139701843), ('飞行', 0.362668514251709)]
狗 的相似词： [('猫', 0.7484437823295593), ('鸟', 0.3281325697898865), ('忠诚', 0.21707698702812195), ('飞翔', 0.20640970766544342), ('在', 0.17947092652320862)]
天空 的相似词： [('水中', 0.6922759413719177), ('飞机', 0.5342141389846802), ('鸟', 0.3054409623146057), ('生活', 0.2645106315612793), ('喜欢', 0.14344635605812073)]
宠物 的相似词： [('可爱', 0.8576368689537048), ('忠诚', 0.7224279046058655), ('小猫', 0.5098384618759155), ('小狗', 0.4703502058982849), ('都', 0.23543407022953033)]
```

图 13　相似词查询函数与四个查询词的实际输出

```python
trained_compare_words = ["猫", "狗", "鸟", "飞机"]
trained_ids = [word_to_id[w] for w in trained_compare_words]
trained_vectors = embeddings[trained_ids].cpu().numpy()
trained_onehot = np.eye(len(vocab), dtype=float)[trained_ids]
trained_comparison = compare_representations(trained_compare_words, trained_onehot, trained_vectc
print(trained_comparison.round(4).to_string(index=False))
trained_comparison.to_csv("output/task4_trained_comparison.csv", index=False, encoding="utf-8-sig
```

词对	独热_欧氏距离	独热_余弦相似度	分布式_欧氏距离	分布式_余弦相似度
猫–狗	1.4142	0.0	4.2782	0.7484
猫–鸟	1.4142	0.0	5.8127	0.4791
猫–飞机	1.4142	0.0	6.3371	0.3546
狗–鸟	1.4142	0.0	5.9453	0.3281
狗–飞机	1.4142	0.0	6.4825	0.1668
鸟–飞机	1.4142	0.0	6.6240	-0.0466

图 14　同一组四个词的独热表示与 Skip-gram 表示对照输出

7　结果分析

7.1　上下文关系与语义关系

“猫”和“狗”在语料中都与“喜欢、吃”等词相邻，并共同出现在“猫和狗都是宠物”中。共享上下文为模型学习相近表示提供了条件，因此二者具有较高的余弦相似度。

“天空”和“水中”在日常语义上并不相似，但本次查询中二者相似度较高。检查语料可见，“天空”出现在“在天空飞翔”和“在天空飞行”中，“水中”出现在“生活在水中”中，它们都与“在”相邻。这种有限的上下文重合可能影响了结果，但仅凭本次训练无法确定各因素的具体作用。

“鱼”和“水中”虽然出现在同一句话中，但在“鱼生活在水中”里，两者不属于窗口大小 1 下的直接邻居。它们的现实关联并没有以直接训练样本的形式输入模型，这也是解释该结果时需要考虑的因素。

因此，词向量相似性反映的是训练条件下的上下文或用法关系，不等同于词典意义上的同义关系。

任务 1 的 SVD 与本次 Skip-gram 使用了不同语料、不同目标和不同维数。不同空间中的向量尺度也不一致，因此不能通过原始欧氏距离大小直接判断两种方法的优劣。

7.2　PCA 二维可视化

使用主成分分析（PCA）将 23 个 10 维词向量投影到二维空间，并在散点旁标注词语，结果如图15所示。

图中，“飞翔”和“飞行”位置较近，“天空”和“水中”也出现在相邻区域，这与部分上下文关系相呼应。“猫”和“狗”虽然在原始空间中具有较高余弦相似度，但在二维图中没有完全聚合到一起。

PCA 保留的是部分方差较大的方向，会丢失其他维度的信息；二维欧氏距离与原始空间中的余弦相似度也不是同一个指标。因此，可视化适合辅助观察，判断相似关系时仍应结合原始词向量的计算结果。

7.3　小语料实验的局限性

本次训练只有 8 个句子、37 个词次，其中部分词只出现一次，能提供的上下文信息较少。模型可以拟合这些训练样本，但一些相似词排名容易受到个别句子和随机初始化影响。

实验没有设置独立测试集，也没有在真实下游任务中评价词向量。因此，本次结果主要用于理解样本构造、训练和相似度分析流程，不能直接用于判断词向量在文本分类或其他应用中的性能。

后续如果继续实验，可以增加自然语料，并在相同条件下比较不同窗口或多次随机初始化的结果，检查观察到的相似关系是否稳定。

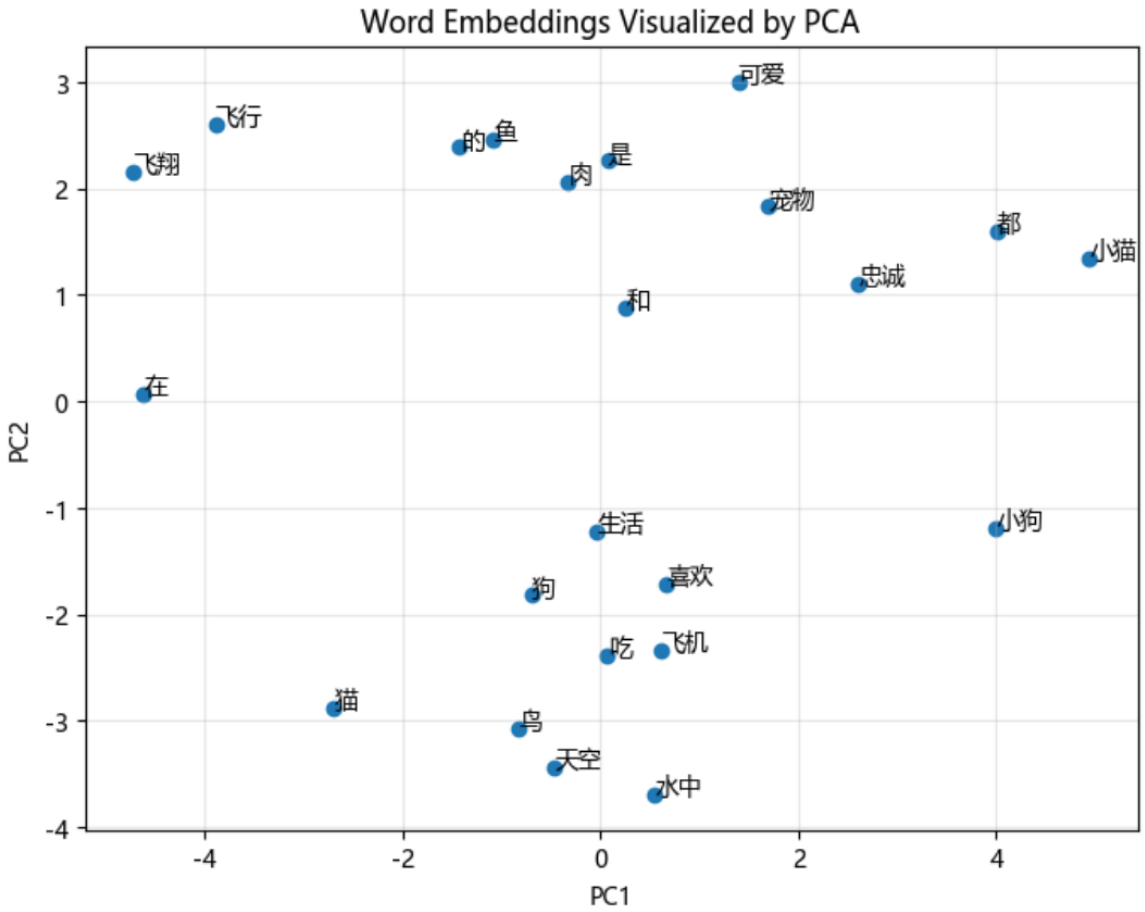


图 15　23 个 Skip-gram 词向量的 PCA 二维投影

8　Word2Vec、GloVe、ELMo 比较

8.1　不同表示方法的区别

表 10　不同词表示方法的基本区别

方法	表示类型	主要信息来源	静态	主要特点
独热表示	稀疏离散表示	词表编号	是	易于构造，不能表达词语相似性
Word2Vec	稠密分布式表示	局部上下文预测	是	通过 CBOW 或 Skip-gram 任务学习词向量
GloVe	稠密分布式表示	全语料累计的词—上下文共现统计	是	根据共现统计学习词表示
ELMo	上下文相关表示	双向语言模型	否	同一词在不同语境中可以得到不同表示

Word2Vec 通过预测任务学习词向量：CBOW 使用上下文预测中心词，Skip-gram 使用中心词预测上下文词 [2]。GloVe 则对整个语料中累计的共现信息建立学习目标；其中的“全局”是指统计信息的汇总，并不意味着完全不使用上下文窗口 [3]。

传统 Word2Vec 和 GloVe 的静态表示为同一个词保存固定向量，难以充分区分该词在不同句子中的含义。ELMo 从双向语言模型中提取与语境相关的表示，例如同一个“苹果”在水果和电子产品相关句子中可以具有不同表示，从而缓解一词多义问题 [4]。

本次实现了小型 Skip-gram 模型；GloVe 和 ELMo 仅进行原理比较，没有运行或训练这两种模型。

8.2 完整 Softmax 与负采样

本实验使用完整 Softmax。对于一个中心词，需要计算整个词表中所有候选上下文词的输出分数，并进行归一化。词表只有 23 个词时，计算成本较低；词表增大后，输出层的计算和更新成本也会增加。

负采样采用不同的训练目标：将真实中心词—上下文词组合作为正例，采样若干词形成负例，再训练模型区分这些组合。这样，每个样本只涉及正例和少量负例，无需每次计算整个词表 [5]。

8.3 层次 Softmax 与哈夫曼树

层次 Softmax 将词组织为树的叶节点，通过从根节点到目标词的路径计算概率。若树较平衡，路径长度约为 $\log_2 V$，因此可以减少相对于完整输出层的计算量，其中 V 为词表大小。

使用哈夫曼树时，高频词对应较短路径，低频词对应较长路径，从而减少按词频加权的平均路径长度。它并不保证每个词的路径都比平衡二叉树更短，其优势在于考虑了词语出现频率 [5]。

本次对负采样、层次 Softmax 和哈夫曼树进行了原理分析，没有添加对应实现，实际 Skip-gram 训练仍采用完整 Softmax。

9 问题记录与解决方法

9.1 PyTorch 缺失与内核环境问题

第一次运行时，导入 torch 出现 ModuleNotFoundError。随后尝试安装时又出现 No matching distribution found，模型定义和训练因此未能执行，后续产生了 nn、model、embeddings 等变量未定义的连锁错误。

排查时先确认 Python 环境和 Jupyter 内核。在已有的 Python 3.10 NLP 环境中安装 CPU 版 PyTorch，并切换到对应内核后，重新运行导入、模型定义和训练代码，最终成功获得损失曲线与词向量矩阵。

这一过程说明，终端环境和 notebook 内核需要对应。依赖导入失败时，应先解决首个错误，再从相关前置单元格开始重新执行。

9.2 PCA 图中文字体缺失

首次绘制 PCA 图时，Matplotlib 提示缺少中文字形，图中的词语显示为方框。原因是默认字体未覆盖相应中文字符。

在绘图前设置 Microsoft YaHei、SimHei 为候选字体，并调整负号显示设置后，重新运行绘图单元格，中文标签恢复正常。最终使用修复后的图片记录可视化结果。

9.3 修改语料后的结果覆盖

修改任务 1 语料时，重新计算使用了相同变量名，原始共现矩阵和后续计算变量容易被覆盖，影响修改前后的比较。

整理结果时，将原始输出与修改后的输出分别记录，在报告中明确标注语料条件，并使用同一词对、同一指标进行比较。两组结果的截图分别对应第 3 节中的原始实验和修改语料实验，避免了结果混淆。

10　实验总结

本实验完成了独热表示与统计式低维表示的比较、上下文窗口样本构造、Skip-gram 模型训练、词对相似度计算和相似词查询，并进行了 PCA 可视化及 CBOW 样本构造。

通过修改一条关于“狗”的短语，可以观察到上下文变化会影响词表示；通过 Skip-gram 训练，可以了解词向量如何作为模型参数从预测任务中学习。相似词查询中既出现了符合直觉的关系，也出现了需要结合语料解释的结果，使我认识到损失下降、词语现实关联和词向量相似度之间并不能直接画等号。

本次使用的语料规模较小，实验重点在于理解实现流程和分析方法。判断词向量质量时，还需要考虑语料规模、词频、窗口大小及评价方式，而不能只依据一次训练结果下结论。

参考文献

[1] 教师提供的课程实验材料：*NLP* 实验 *2*：词向量的学习与分析，实验设计 Notebook。

[2] Mikolov T, Chen K, Corrado G, Dean J. Efficient Estimation of Word Representations in Vector Space. 2013. arXiv:1301.3781.

[3] Pennington J, Socher R, Manning C D. GloVe: Global Vectors for Word Representation. EMNLP, 2014: 1532–1543. DOI:10.3115/v1/D14-1162.

[4] Peters M E, Neumann M, Iyyer M, et al. Deep Contextualized Word Representations. NAACL-HLT, 2018: 2227–2237. DOI:10.18653/v1/N18-1202.

[5] Mikolov T, Sutskever I, Chen K, Corrado G, Dean J. Distributed Representations of Words and Phrases and their Compositionality. 2013. arXiv:1310.4546.